p.33 해설 — 컴퓨팅 사고력이란?

> 왼쪽 원문을 읽으며 밑줄 칠 곳: **문단별 핵심 문장**을 아래 순서대로 찾아보세요.

① **첫 문단 (왼쪽 단 위)** — 컴퓨팅 사고력은 *사람이든 기계든* 계산 과정의 **힘과 한계**에서 출발한다. 저자는 세 가지 질문을 던진다: 사람이 더 잘하는 것은? 컴퓨터가 더 잘하는 것은? 그리고 가장 근본적으로 **"무엇이 계산 가능한가?"** → 🔎 *What is computable?* 에 밑줄. (계산 가능성 이론, 튜링)

② **둘째 문단** — 이 글의 **핵심 주장**: 읽기·쓰기·셈하기(3R)에 컴퓨팅 사고력을 더해 **모든 아이의 기본 능력**으로 삼아야 한다. → 🔎 *fundamental skill for everyone* 에 밑줄.

③ **셋째 문단** — 정의: 컴퓨터 과학의 개념을 바탕으로 **문제 해결 · 시스템 설계 · 인간 행동 이해**를 하는 것.

④ **넷째~다섯째 문단 (오른쪽 단 위)** — 문제를 만나면 "얼마나 어려운가? 가장 좋은 방법은?"을 묻는다. 근사해로 충분한지, **무작위성**을 쓸 수 있는지, 오류(거짓 양성/음성)를 허용할지 따진다. 어려운 문제를 **이미 풀 줄 아는 문제로 바꾸기** (환원·변환·시뮬레이션). → 수학의 "환원(reduction)"과 같은 발상.

⑤ **"Computational thinking is …" 나열** — 재귀적 사고, 병렬 처리, 코드와 데이터의 상호 해석, **타입 검사 = 차원 분석의 일반화**, 별칭(aliasing)의 장단점, 간접 주소의 비용과 힘, 프로그램의 **아름다움과 단순함**까지 평가하기.

⑥ **마지막 문단** — **추상화와 분해**, 관심사의 분리, 적절한 표현 고르기, **불변식(invariant)** 으로 시스템 설명하기, 세부를 다 몰라도 큰 시스템을 안전하게 다루는 자신감.

핵심 용어	뜻
reduction	환원 (아는 문제로 바꾸기)
abstraction / decomposition	추상화 / 분해
invariant	불변식
aliasing	하나에 여러 이름 붙이기

p.34 해설 — 일상 속 컴퓨팅 사고력

① **왼쪽 단 위 (앞 쪽에서 이어짐)** — 여러 사용자를 예상한 **모듈화**, 미래 사용을 대비한 **프리페칭·캐싱**. 최악의 상황에 대비한 **중복·피해 격리·오류 정정**. 교통 정체를 "교착 상태(deadlock)"로, 약속을 "인터페이스"로 보는 눈.

② **발견적 추론과 탐색** — 불확실성 속의 계획·학습·스케줄링. **"탐색, 탐색, 또 탐색"** — 웹 검색 결과, 게임 전략, 반례 찾기 모두 탐색. **시간과 공간의 트레이드오프**. → 🔎 기계학습과 연결: "학습 = 가설 공간의 탐색"

③ **일상 예시 (왼쪽 단 아래)** ⭐ **수업에 바로 쓰기 좋음**

일상 상황	컴퓨팅 개념
등교 전 가방 챙기기	프리페칭과 캐싱
잃어버린 장갑 찾으러 왔던 길 되짚기	백트래킹
스키를 언제까지 빌리고 언제 살까?	온라인 알고리즘
마트에서 어느 줄에 설까?	다중 서버 성능 모델링
정전에도 전화가 되는 이유	고장 독립성과 중복 설계
CAPTCHA	어려운 AI 문제를 보안에 활용

④ **다른 학문에 미친 영향 (오른쪽 단)** — **기계학습이 통계학을 바꿨다**: 데이터 크기와 차원의 규모가 달라짐. 생물학(단백질 구조 표현), 경제학(계산 게임 이론), 화학(나노컴퓨팅), 물리학(양자 컴퓨팅)도 변화 중. → 🔎 *machine learning has transformed statistics* 에 밑줄 — 이번 주 Zhang·Belkin 논문이 바로 그 사례!

⑤ **비유** — 유비쿼터스 컴퓨팅이 "어제의 꿈 → 오늘의 현실"이었듯, 컴퓨팅 사고력은 **내일의 현실**이 될 것이다.

⑥ **"What It Is, and Isn't"** — 다음 쪽의 6가지 특징을 예고. 컴퓨터 과학 = **무엇을 계산할 수 있고 어떻게 계산하는가**에 대한 학문.

p.35 해설 — 무엇이고 무엇이 아닌가

① 6가지 특징 (왼쪽 단, 이탤릭 제목들) — 각 제목에 밑줄 치고 번호를 붙여 보세요.

#	원문 제목	의미
1	Conceptualizing, not programming	코딩이 아니라 **여러 수준의 추상화**로 생각하기
2	Fundamental, not rote skill	기계적 반복이 아닌 **기본 소양**
3	A way that humans, not computers, think	사람을 컴퓨터처럼 만드는 게 아니라 **사람의 문제 해결 방식**
4	Complements and combines mathematical and engineering thinking	**수학적 사고 + 공학적 사고**의 결합
5	Ideas, not artifacts	제품이 아니라 **개념**
6	For everyone, everywhere	너무 자연스러워 **의식하지 않게 될 때** 완성

→🔎 4번이 특히 중요: 컴퓨터 과학의 기초는 수학이지만, **실제 기계의 제약** 때문에 "수학적으로만"이 아니라 "계산적으로" 생각해야 한다는 점. (예: Adam 논문의 ϵ — 수학엔 없고 계산엔 필요한 값)

② 오른쪽 단 위 — 사람들은 컴퓨터 과학 = 프로그래밍이라고 오해하고, 진로가 좁다고 생각한다. 그래서 **초중고 학생·교사·학부모**에게 두 메시지를 전해야 한다:

- 아직 풀리지 않은 **흥미로운 과학 문제**가 많다
- 컴퓨터 과학을 전공해도 **무엇이든 할 수 있다** (의학, 법, 경영, 예술까지)

③ 마지막 문단 — 저자의 제안 — 대학 신입생(비전공자 포함)에게 「컴퓨터 과학자처럼 생각하는 법」 강좌를 열고, **대학 입학 전 학생들에게 계산적 방법과 모델을 접하게 하자**. →🔎 *pre-college students* 에 밑줄 — **ai_life 사이트의 존재 이유와 정확히 같은 주장!**

✍️ **교사로서 생각해볼 것**

1. ai_life의 5개 단원은 위 6가지 특징 중 어떤 것을 가장 잘 보여주는가?
2. "Conceptualizing, not programming" — 코딩 없이 컴퓨팅 사고력을 가르치는 활동을 하나 설계해 보자.